

Licenciatura em Engenharia Informática – EaD



RELATÓRIO TÉCNICO: Aplicação Móvel para Registo de Hábitos de Saúde e Bem-Estar (Habittus)

Licenciatura em Engenharia Informática – EaD

Grupo 11

Rui Filipe da Silva Encarnação (20240458)
Daniel Chambel da Cruz (20240455)
Cátia Alexandra Ferreira Macedo (20240609)
Carlos Daniel Pinto Vieira (20240964)
Carlos Manuel Ferreira Dias (20241639)
Marcelo Agostinho Lopes (20241915)
Rute Maria da Fonseca Gouveia Frutuoso (20241693)

Lisboa, dia 29 de Janeiro de 2026

Índice

Resumo	1
Introdução	2
1. Modelação e Arquitetura.....	3
1.1. Modelação Conceptual	3
1.2. Arquitetura da Aplicação/Diagrama de componentes	5
2. Metodologia.....	7
3. Resultados	13
4. Considerações Finais.....	17
4.1. Modelação	17
4.2. Implementação	17
4.2.1. Programação Dispositivos Moveis	17
4.2.2. Programação Orientada a Objetos	17
4.2.3. Base de dados.....	18
4.3. Desenvolvimento	18
4.4. Síntese e Perspetivas de Evolução	18
Anexos	20
Diagrama ER completo	21
Diagrama ER simplificado	22
Diagrama UML.....	23
Referências bibliográficas	24

Índice de Figuras

Figura 1 – Diagrama ER do Sistema Habittus.....	4
Figura 2 – Exemplo de Registo em water_intake	4
Figura 3 – Exemplo de registo em sleep_session.....	4
Figura 4 – Exemplo de registo em meal.....	5
Figura 5 – Exemplo de registo em activity	5
Figura 6 – Diagrama de Classes UML	7
Figura 7 – Ecrã de autenticação da aplicação Habittus.....	14
Figura 8 – Ecrã principal da aplicação Habittus.....	15
Figura 9 – Ecrã de registo e acompanhamento de hábitos	16

Resumo

O presente relatório consolida o desenvolvimento do projeto *Habittus*, realizado no âmbito das Unidades Curriculares de Programação Orientada a Objetos, Programação de Dispositivos Móveis e Bases de Dados. O objetivo principal foi conceber e implementar uma aplicação móvel destinada ao registo e monitorização de hábitos de saúde e bem-estar, suportada por uma base de dados relacional e por uma arquitetura orientada a objetos.

A aplicação permite o registo estruturado de ingestão de água, refeições, sono, atividade física, estados de humor, definição de metas e lembretes, assegurando integridade dos dados, separação de responsabilidades e preparação para evolução futura. A solução recorre a Dart e Flutter para a implementação da aplicação, a uma lógica orientada a objetos modular e a PostgreSQL/Supabase para persistência e autenticação.

O desenvolvimento seguiu uma abordagem incremental e iterativa, com validações frequentes face aos requisitos funcionais e aos mockups definidos no Figma, levantando desafios de requisitos versus operacionalidade na área de bases de dados.

Os resultados evidenciam coerência entre modelação conceptual, implementação lógica e interface móvel, cumprindo os objetivos pedagógicos definidos.

Introdução

O projeto *Habittus* surge no contexto de um trabalho interdisciplinar que integra as Unidades Curriculares de Programação Orientada a Objetos, Programação de Dispositivos Móveis e Bases de Dados. O seu objetivo é o desenvolvimento de uma aplicação móvel que permita ao utilizador registar, acompanhar e analisar rotinas associadas à saúde e ao bem-estar. Estas rotinas incluem hidratação, sono, alimentação, atividade física, humor, hábitos e, para utilizadoras do sexo feminino, o ciclo menstrual.

A aplicação foi concebida com base em boas práticas de engenharia de software, garantindo alinhamento entre a modelação de dados, a lógica orientada a objetos e a implementação móvel. Este relatório apresenta de forma sintética a arquitetura adotada, a metodologia aplicada e os resultados obtidos, assegurando consistência terminológica e evitando duplicações entre disciplinas.

1. Modelação e Arquitetura

1.1. Modelação Conceptual

A modelação conceptual do sistema baseia-se na identificação das principais entidades do domínio e das respetivas relações. A entidade central é User, à qual se associam os diversos registos de hábitos e entidades de suporte.

O modelo conceptual foi definido de forma a suportar diferentes domínios funcionais da aplicação, garantindo consistência entre o utilizador, os registos diários e as métricas associadas às várias áreas de acompanhamento da saúde. Este modelo foi também estruturado respeitando princípios de normalização, evitando redundâncias e assegurando coerência entre a modelação e a implementação.

As principais entidades incluem: User, Profile, WaterIntake, Meal, MealItem, Food, SleepSession, Activity, MoodEntry, Goal, Reminder, Habit e HabitLog. As relações seguem maioritariamente cardinalidades do tipo 1:N, garantindo integridade referencial e coerência semântica. Algumas associações específicas, como a relação entre Meal e Food, exigiram uma relação N:N, resolvida através da entidade MealItem.

A correspondência entre o modelo Entidade-Relacionamento e o modelo orientado a objetos é direta, permitindo uma transposição consistente para a lógica da aplicação. As entidades definidas no modelo conceptual encontram correspondência direta nas classes da aplicação e nos ecrãs funcionais, assegurando coerência entre a base de dados, a lógica orientada a objetos e a interface móvel.

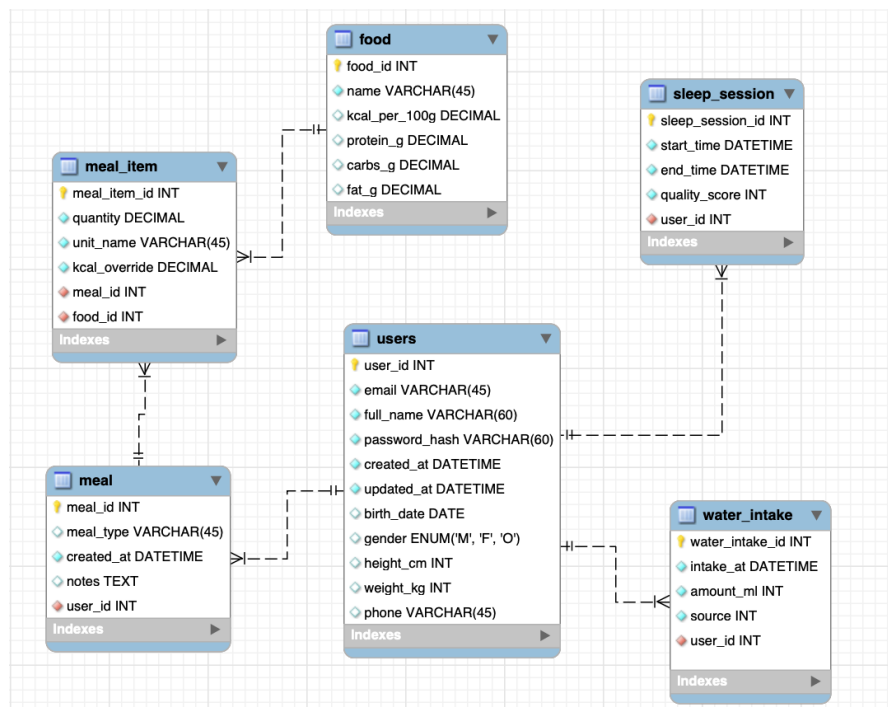


Figura 2– Diagrama Entidade-Relacionamento do sistema Habittus, representando as entidades principais, os respectivos atributos e as relações que suportam o registo estruturado de hábitos de saúde e bem-estar.

Foram criados registos de teste com o objetivo de validar o modelo relacional, assegurando integridade referencial, cardinalidades corretas e coerência dos dados por utilizador.

water_intake_id i..	intake_at timestamptz	amount_ml i..	source water_source_enum	user_id i..
1	2026-01-25 00:00:00+00	1000	manual	7
2	2026-01-24 00:00:00+00	1750	manual	7

Figura 2 – Exemplo de registo de consumo de água tabela water_intake.

sleep_session_id i...	start_time timestamptz	end_time timestamptz	quality_score int4	user_id i..	sleep_date timestamptz	duration_rr
1	2026-01-25 11:15:00+00	2026-01-25 19:15:00+00	3	7	2026-01-25 00:00:00+00	480
2	2026-01-26 11:45:00+00	2026-01-26 19:45:00+00	3	13	2026-01-26 00:00:00+00	480

Figura 3 – Exemplo de registo de horas de sono na tabela sleep_session.

 meal_id i. v	meal_type varchar v	created_at timestampz v	notes text v	 user_id i.. v
2	Almoço	2026-01-25 00:00:00+00	NULL	7 
3	Almoço	2026-01-24 00:00:00+00	NULL	7 

Figura 4 – Exemplo de registo de refeições na tabela meal.

 activity_id i. v	user_id i.. v	 activity_type_id int4 v	duration_min int4 v	steps int4 v	kcal int4 v	created_at timestampz v	notes text
3		3 	10	NULL	NULL	2026-01-25 19:53:00+00	NULL
4		2 	60	NULL	100	2026-01-24 19:54:00+00	NULL

Figura 5 – Exemplo de registo de atividade física na tabela activity_type.

Este conjunto de dados confirma que todas as tabelas dependentes mantêm ligação direta à tabela user, assegurando integridade referencial, coerência estrutural e suporte à escalabilidade futura do sistema.

1.2. Arquitetura da Aplicação/Diagrama de componentes

A aplicação adota uma arquitetura em camadas, promovendo a separação clara de responsabilidades. A arquitetura adotada segue uma abordagem próxima do padrão **MVVM**, garantindo separação entre interface, lógica de apresentação e acesso a dados, facilitando manutenção e testes.

A camada de interface é implementada em Flutter (SDK 3.x stable) e responsável pela apresentação e navegação entre ecrãs. Por sua vez, a camada lógica da aplicação integra controladores que aplicam as regras de negócio definidas pelos modelos orientados a objetos desenvolvidos em **Dart (SDK 3.x)**. A gestão de estados é assegurada com **Provider (6.0.5)**, que implementa o padrão Observer, permitindo que a UI reaja automaticamente às alterações dos controladores.

A camada de dados é estruturada com repositórios, que abstraem o acesso à base de dados e asseguram um fluxo unidirecional de dados entre a interface, a lógica de controlo e a persistência. O backend foi assegurado pelo **Supabase**, utilizando a biblioteca **supabase_flutter (2.3.0)** para autenticação, acesso à base de dados **PostgreSQL** integrada e sincronização em tempo real.

Para complementar as funcionalidades da interface, foram utilizadas bibliotecas adicionais, como **FL Chart (1.1.1)**, responsável pela geração dos gráficos interativos utilizados nos módulos de hidratação, sono e atividade física, e o **Cupertino Icons (1.0.8)**, que disponibiliza um conjunto de ícones no estilo iOS, garantindo consistência visual na interface.

Para além da organização em camadas, a arquitetura do sistema integra diversos padrões de desenho, nomeadamente: o **Singleton** (`core/database/supabase_service.dart`), que garante a existência de uma única instância do serviço de base de dados em toda a aplicação; o **Repository** (`repositories/interfaces/repository_interfaces.dart`), que abstrai a camada de acesso a dados e facilita a substituição de implementações; o **Factory** (`factories/model_factory.dart`), responsável pela criação centralizada de modelos com valores padrão e validações; o **Observer** (`ChangeNotifier + Provider`), que notifica a UI automaticamente quando os dados mudam; e o **Strategy** (`strategies/validation_strategy.dart`), para definir diferentes estratégias de validação e cálculo que podem ser alteradas em tempo de execução.

A comunicação entre camadas segue um fluxo unidirecional e bem definido: dados fluem da interface para os controladores, destes para os repositórios e, finalmente, para a camada de persistência. Esta organização, em conjunto com padrões de desenho aplicados, reforça a modularidade da arquitetura e garante maior testabilidade, facilidade de manutenção e evolução futura do sistema.

Planeamento e Execução por Sprints

Sprint 1 – Estrutura Base (Semanas 3–4)

Objetivo: Estabelecer a base estrutural da aplicação e criar componentes reutilizáveis.

Foram definidas e concluídas as seguintes user stories:

- **US01:** Como developer, quero uma estrutura de pastas organizada;
- **US02:** Como utilizador, quero um menu de navegação consistente;
- **US03:** Como utilizador, quero uma AppBar reutilizável.

Entregáveis:

- HabittusAppBar – barra superior com menu e acesso ao perfil;
- HabittusDrawer – menu lateral de navegação;
- Estrutura de pastas organizada segundo o padrão MVVM.

Sprint 2 – DatePills e Hidratação (Semanas 5–6)

Objetivo: Implementar um sistema de seleção de datas responsivo e o ecrã de registo de hidratação.

User stories concluídas:

- **US04:** Como utilizador, quero seleccionar dia, mês e ano de forma independente;
- **US05:** Como utilizador, quero registar copos de água;
- **US06:** Como utilizador, quero visualizar um gráfico semanal de hidratação.

Entregáveis:

- Componente DatePills responsivo com recurso a LayoutBuilder;
- WaterIntakeScreen com grelha 4×4;
- WaterController e WaterRepository.

Sprint 3 – Humor e Sono (Semanas 7–8)

Objetivo: Implementar funcionalidades de registo de humor e sono.

User stories concluídas:

- **US07:** Como utilizador, quero registar o meu estado de humor;
- **US08:** Como utilizador, quero registar a qualidade do sono;
- **US09:** Como utilizador, quero registar as horas de sono.

Entregáveis:

- MoodInquiryScreen com formulário multi-step;
- SleepTimeScreen com slider de duração;
- MoodController e SleepController.

Sprint 4 – Alimentação (Semanas 9–10)

Objetivo: Implementar o registo de refeições com base numa base de dados de alimentos.

User stories concluídas:

- **US10:** Como utilizador, quero registar refeições;
- **US11:** Como utilizador, quero pesquisar alimentos;
- **US12:** Como utilizador, quero visualizar as calorias consumidas.

Entregáveis:

- MealsScreen com listagem de refeições;
- AddMealScreen com pesquisa de alimentos;
- Base de dados com 328 alimentos;
- MealController com mecanismo de cache otimizado.

Sprint 5 – Atividade Física (Semana 11)

Objetivo: Implementar o registo de atividades físicas.

User stories concluídas:

- **US13:** Como utilizador, quero registar atividades físicas;
- **US14:** Como utilizador, quero visualizar calorias queimadas;
- **US15:** Como utilizador, quero consultar um gráfico semanal de atividade.

Entregáveis:

- PhysicalActivityScreen;
- AddActivityScreen;
- WeeklyActivityChart.

Sprint 6 – Hábitos e Dashboard (Semana 12)

Objetivo: Implementar o sistema de hábitos e um dashboard central.

User stories concluídas:

- **US16:** Como utilizador, quero criar hábitos positivos e negativos;
- **US17:** Como utilizador, quero acompanhar streaks de hábitos;
- **US18:** Como utilizador, quero um dashboard com resumo diário.

Entregáveis:

- HabitsScreen com listagem de hábitos;
- AddHabitScreen;
- HomeDashboardScreen com cartões agregadores de todas as áreas.

Sprint 7 – Ciclo Menstrual e Autenticação (Semana 13)

Objetivo: Implementar funcionalidades específicas de ciclo menstrual e autenticação.

User stories concluídas:

- **US19:** Como utilizadora, quero registrar o ciclo menstrual;
- **US20:** Como utilizadora, quero registrar sintomas;
- **US21:** Como utilizador, quero autenticar-me na aplicação;
- **US22:** Como utilizador, quero registrar-me na aplicação.

Entregáveis:

- MenstrualCycleScreen (disponível apenas para utilizadores femininos);
- LoginScreen e RegisterScreen;
- ProfileScreen;
- UserController.

Sprint 8 – Integração e Polimento Final (Semana 14)

Objetivo: Realizar a integração global da aplicação, testes e otimizações finais.

Tarefas concluídas:

- Integração dos controllers com o backend Supabase;
- Correção de erros de leitura e gravação de dados;
- Otimização de queries, com especial incidência no MealController;
- Implementação de visibilidade condicional por género;
- Testes de integração;
- Elaboração da documentação técnica e do relatório final.

Ferramentas, Tecnologias e Validação

A prototipagem dos ecrãs foi realizada em **Figma**, permitindo validar antecipadamente os fluxos de navegação antes da implementação. O desenvolvimento foi efetuado maioritariamente no **Visual Studio Code**, recorrendo às extensões oficiais de Flutter e Dart.

No controlo de qualidade, foi utilizado o **Flutter Test (SDK)**, para execução de testes unitários e de widgets, e o **Flutter Lints (6.0.0)**, que assegura boas práticas de programação através de um conjunto de regras de análise estática do código. A aplicação foi testada com o **Android SDK (API 21+)**, em emuladores e dispositivos físicos, assegurando compatibilidade com diferentes dispositivos.

O controlo de versões foi efetuado com **Git** e **GitHub**, permitindo um acompanhamento estruturado e colaborativo da evolução do projeto.

Na fase final, foi dada especial atenção à estabilização do sistema, correção de inconsistências e otimização do acesso a dados, assegurando um funcionamento fiável e consistente da solução. Foram ainda realizados testes funcionais às principais funcionalidades da aplicação, garantindo a conformidade com os requisitos definidos e a correta integração entre a interface, a lógica de negócio e a base de dados. As validações efetuadas permitiram detetar e corrigir inconsistências numa fase precoce, contribuindo para a estabilidade da solução final.

3. Resultados

As funcionalidades essenciais definidas para o projeto encontram-se implementadas e operacionais, incluindo autenticação, registo de hábitos, definição de metas e visualização de progresso. Estas funcionalidades permitiram validar os principais fluxos de utilização definidos na fase de modelação, assegurando uma experiência coerente para o utilizador.

Os resultados evidenciam a implementação consistente de múltiplos módulos funcionais, cobrindo diferentes áreas do acompanhamento de hábitos de saúde, sendo eles: hidratação, sono, alimentação, humor, atividade física, hábitos de ciclo menstrual. Cada módulo apresenta um comportamento estável e integração coerente entre os vários componentes da aplicação.

Do ponto de vista da Programação Orientada a Objetos, foram aplicados os princípios SOLID e diversos padrões de desenho, como Singleton, Factory e Observer, garantindo modularidade, extensibilidade e facilidade de manutenção. A estrutura de classes reflete diretamente o modelo concetual definido, garantindo coerência entre a modelação, a lógica orientada a objetos e a implementação prática.

Na componente de Programação de Dispositivos Móveis, a arquitetura adotada permite uma navegação consistente entre ecrãs e uma gestão eficaz do estado da aplicação, suportada por controladores reativos e mecanismos de atualização automática da interface, contribuindo para a estabilidade e fluidez do comportamento da interface.

Relativamente às Bases de Dados, o modelo final encontra-se normalizado, com integridade referencial assegurada, índices definidos e mecanismos de segurança implementados, contribuindo para a consistência e fiabilidade do armazenamento da informação. A base de dados final suporta um conjunto alargado de entidades interrelacionadas, permitindo o armazenamento estruturado de registos diários, históricos e métricas agregadas, com consultas otimizadas para operações frequentes de leitura e escrita, assegurando eficiência e consistência no acesso à informação.

Os resultados obtidos demonstram que a aplicação responde de forma consistente aos requisitos definidos, evidenciando a correta integração técnica e coerência entre componentes de acordo com as necessidades previstas no enunciado do projeto. As funcionalidades implementadas foram

validadas em cenários de utilização realistas, revelando estabilidade e coerência no comportamento do sistema.

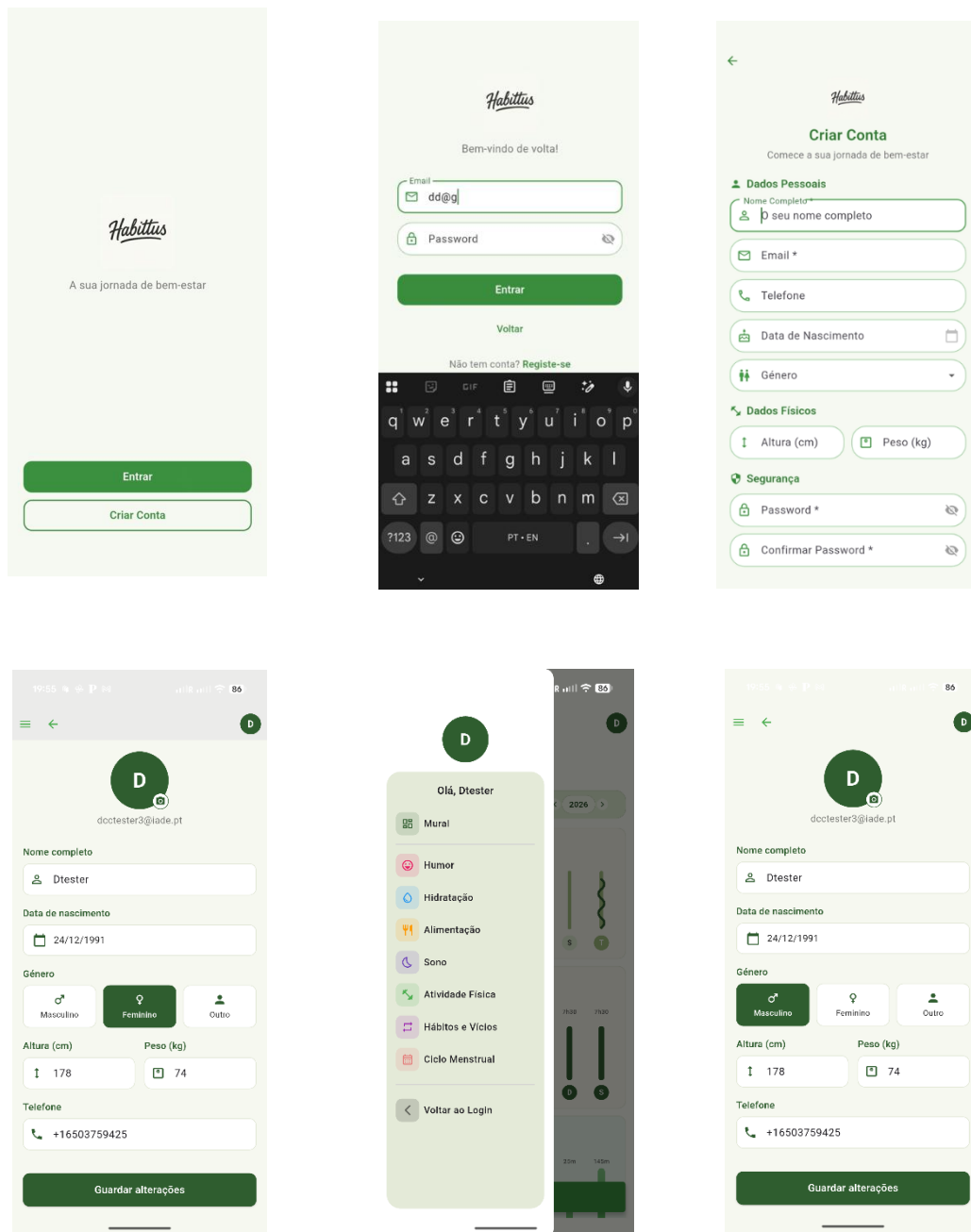


Figura 7 – Ecrã de autenticação da aplicação Habittus, responsável pelo controlo de acesso do utilizador e pela associação segura dos registos de hábitos à sua conta pessoal.

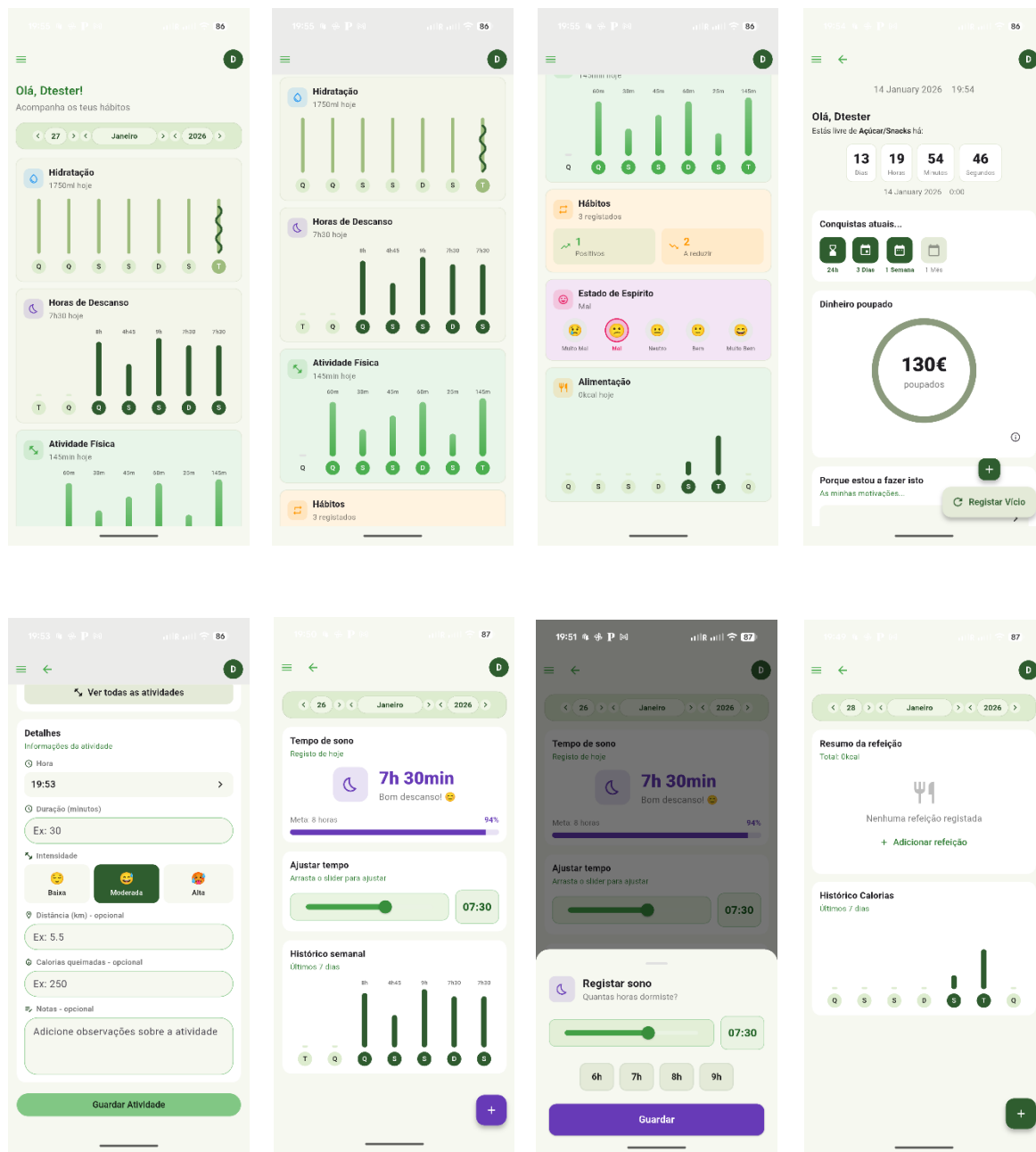


Figura 8 – Ecrã principal (Dashboard) da aplicação Habittus, apresentando o resumo diário dos hábitos registados e permitindo o acesso aos diferentes módulos funcionais

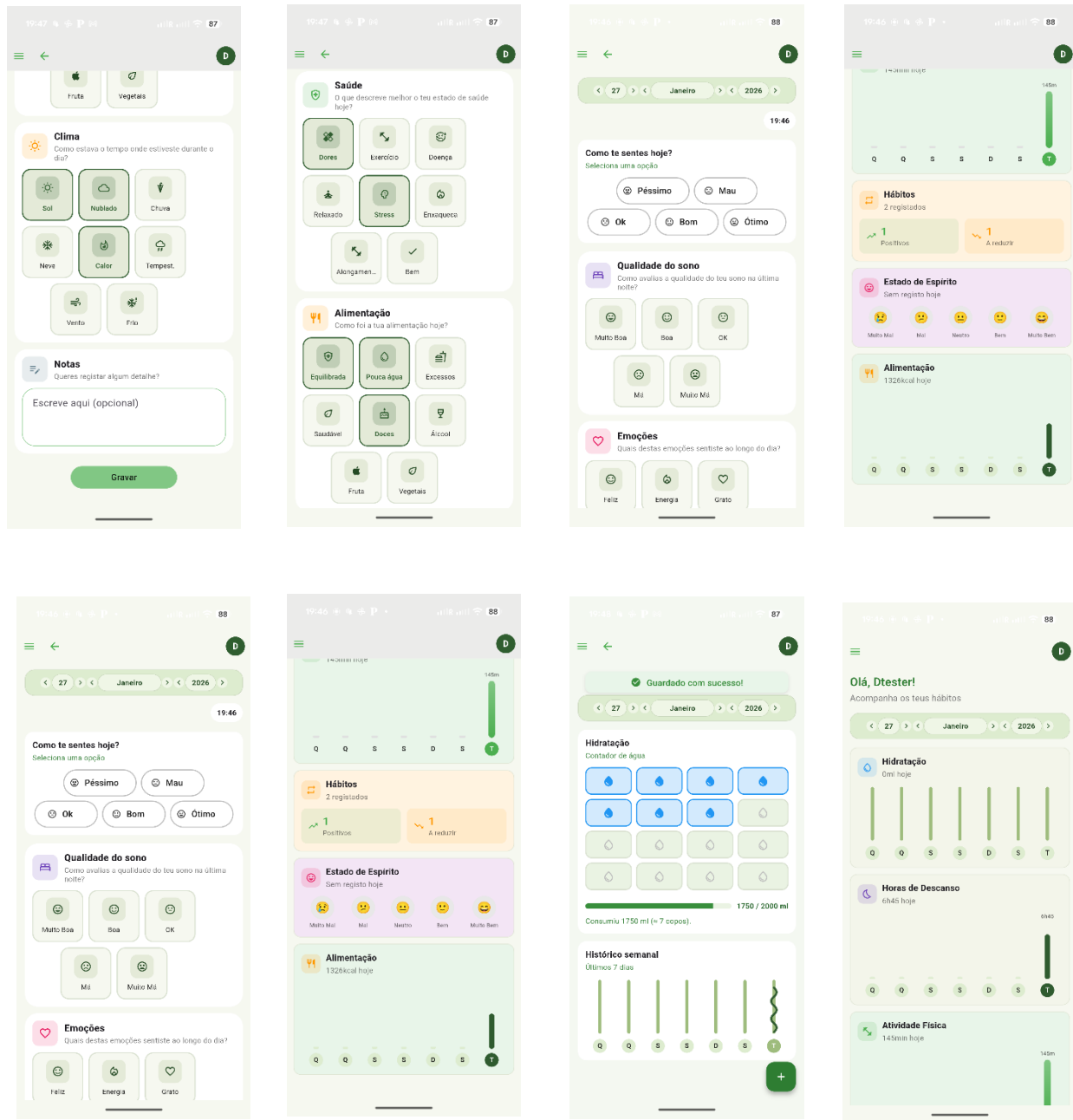


Figura 9 – Ecrã de registo e acompanhamento de hábitos, ilustrando a aplicação prática da lógica orientada a objetos e da persistência de dados na interação com o utilizador.

4. Considerações Finais

O projeto Habittus procura cumprir os requisitos pedagógicos definidos para as três Unidades Curriculares, evidenciando uma integração coerente entre a modelação de dados, a lógica orientada a objetos e o desenvolvimento de aplicações móveis. A solução desenvolvida apresenta uma arquitetura sólida, modular e alinhada com as boas práticas de engenharia de software.

4.1. Modelação

O processo de desenvolvimento iniciou-se com a modelação conceptual e prototipagem em Figma, com base nos requisitos identificados no estudo de mercado realizado. Esta fase permitiu definir fluxos de navegação, componentes de interface e funcionalidades diferenciadoras, incluindo a integração do registo do ciclo menstrual, uma funcionalidade ainda pouco explorada em aplicações de saúde generalistas e com potencial de evolução futura para módulos avançados de acompanhamento e previsão da saúde menstrual.

4.2. Implementação

4.2.1. Programação Dispositivos Moveis

Durante a transição do protótipo para a implementação prática em Flutter, foi necessário proceder a várias decisões de ajuste e racionalização funcional, resultantes de constrangimentos técnicos, temporais e de alinhamento com os objetivos académicos do projeto. Algumas funcionalidades inicialmente previstas, como um menu de configurações avançadas, o registo de notas de voz e a associação de fotografias aos registos, foram reavaliadas e excluídas, por não se revelarem essenciais para o núcleo funcional da aplicação ou por implicarem impactos significativos na estrutura da base de dados e no esforço de implementação. De igual forma, a funcionalidade de contagem de passos foi analisada e considerada dependente da integração direta com sensores dos dispositivos móveis, tendo sido substituída por uma abordagem centrada na geolocalização associada à atividade física, permitindo enriquecer os registos sem comprometer a viabilidade técnica do projeto.

4.2.2. Programação Orientada a Objetos

No âmbito da Programação Orientada a Objetos (POO), a aplicação foi estruturada com base numa modelação rigorosa de classes, refletindo diretamente as entidades definidas no modelo relacional. A aplicação consistente dos princípios fundamentais da POO nomeadamente encapsulamento, herança, polimorfismo e abstração permitiu a construção de uma lógica de negócio coesa,

reutilizável e extensível. A correspondência clara entre o modelo Entidade-Relacionamento e o Diagrama de Classes UML facilitou a separação de responsabilidades, a manutenção do código e a preparação para evoluções futuras.

4.2.3. Base de dados

No contexto da unidade curricular de Bases de Dados, o modelo relacional e respetivos relatórios foram inicialmente desenvolvidos de acordo com o briefing original. Após troca de docente, o briefing foi alterado, obrigando à adaptação do projeto a novos requisitos. Para responder a esta alteração, foi definida uma abordagem mista, assente na correção e adaptação do modelo relacional original para garantir a funcionalidade global da aplicação, em paralelo com o desenvolvimento de um submodelo relacional específico, alinhado com os requisitos formais do novo briefing da unidade curricular. Esta estratégia permitiu assegurar compatibilidade com os dados existentes e minimizar impactos no funcionamento da aplicação. Contudo, tornou-se necessário, cerca de duas semanas antes da entrega, proceder a uma reformulação estrutural significativa, envolvendo ajustes ao nível das tabelas, chaves primárias e estrangeiras, relacionamentos, normalização, scripts DDL/DML e validação das queries existentes. Apesar de mais exigente do ponto de vista técnico e organizacional, esta abordagem revelou-se a única solução viável para garantir o cumprimento simultâneo dos requisitos funcionais da aplicação e dos entregáveis académicos definidos para a unidade curricular.

4.3. Desenvolvimento

Relativamente à gestão do desenvolvimento, foram utilizados dois repositórios GitHub distintos. O primeiro foi dedicado ao desenvolvimento colaborativo do projeto, permitindo o trabalho conjunto de todos os elementos do grupo, com controlo de versões, integração incremental e resolução de conflitos. O segundo repositório foi utilizado para efeitos de controlo documental e processual, concentrando a versão submetida do projeto sob responsabilidade de um único elemento do grupo, assegurando rastreabilidade, consistência dos artefactos finais e conformidade com os requisitos de entrega académica.

4.4. Síntese e Perspetivas de Evolução

Em síntese, o projeto evidencia a capacidade do grupo em avaliar requisitos, tomar decisões técnicas fundamentadas, adaptar soluções em fases avançadas do desenvolvimento e manter coerência entre arquitetura, dados e interface. Para além dos objetivos académicos, o trabalho

desenvolvido constitui uma base sólida para futuras evoluções funcionais, de desempenho e de escalabilidade, nomeadamente na análise histórica de dados, integração de sensores, enriquecimento dos módulos de saúde e otimização global da aplicação.

Anexos

Diagrama ER completo

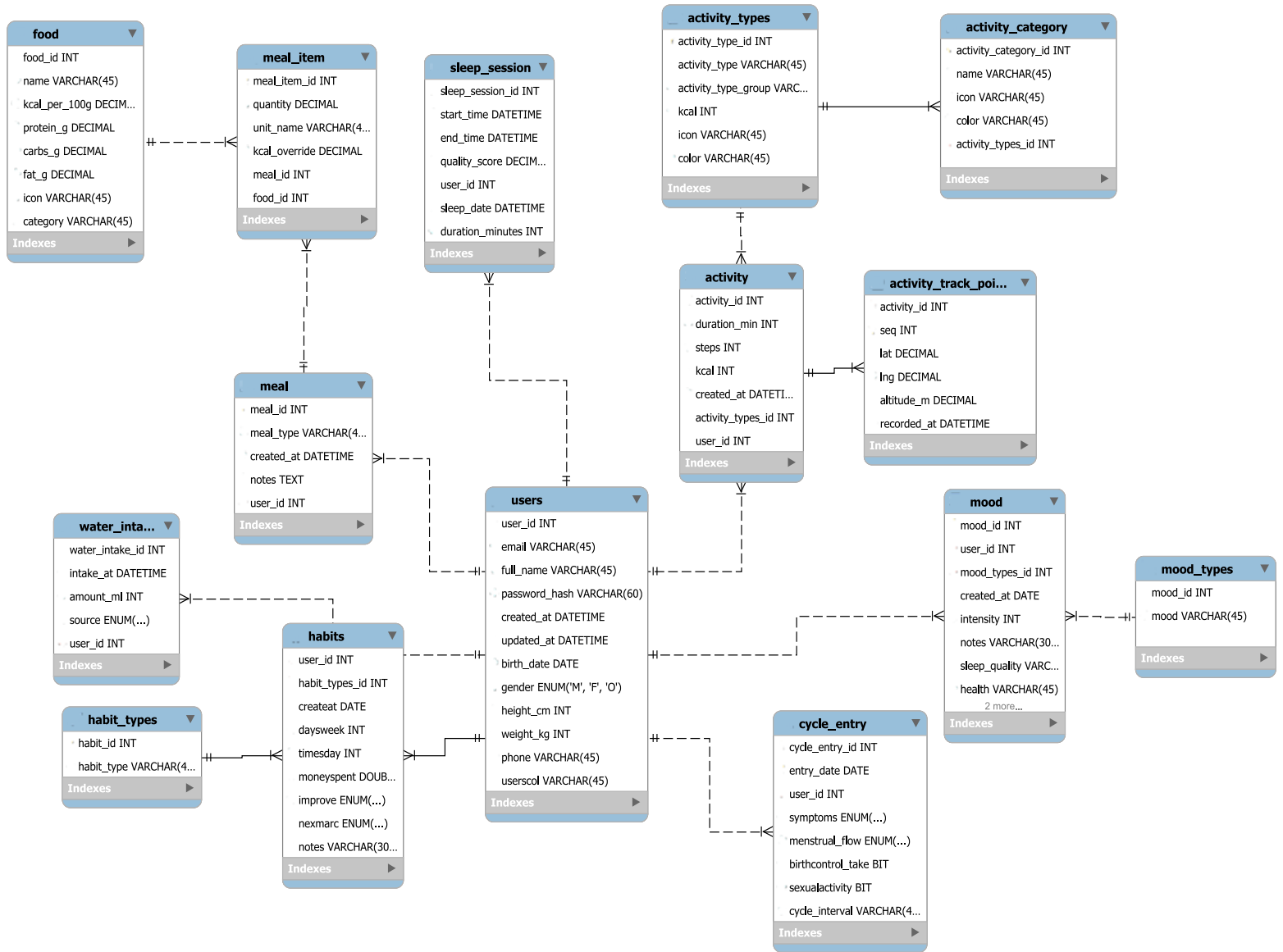


Diagrama ER simplificado

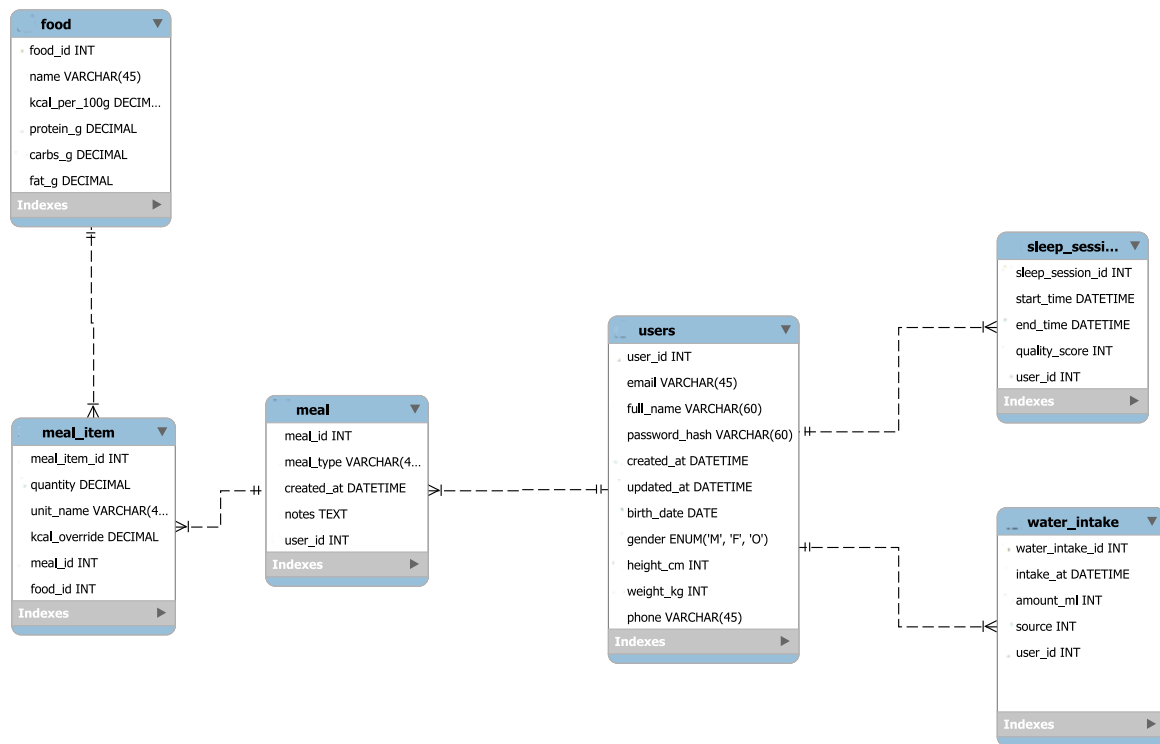
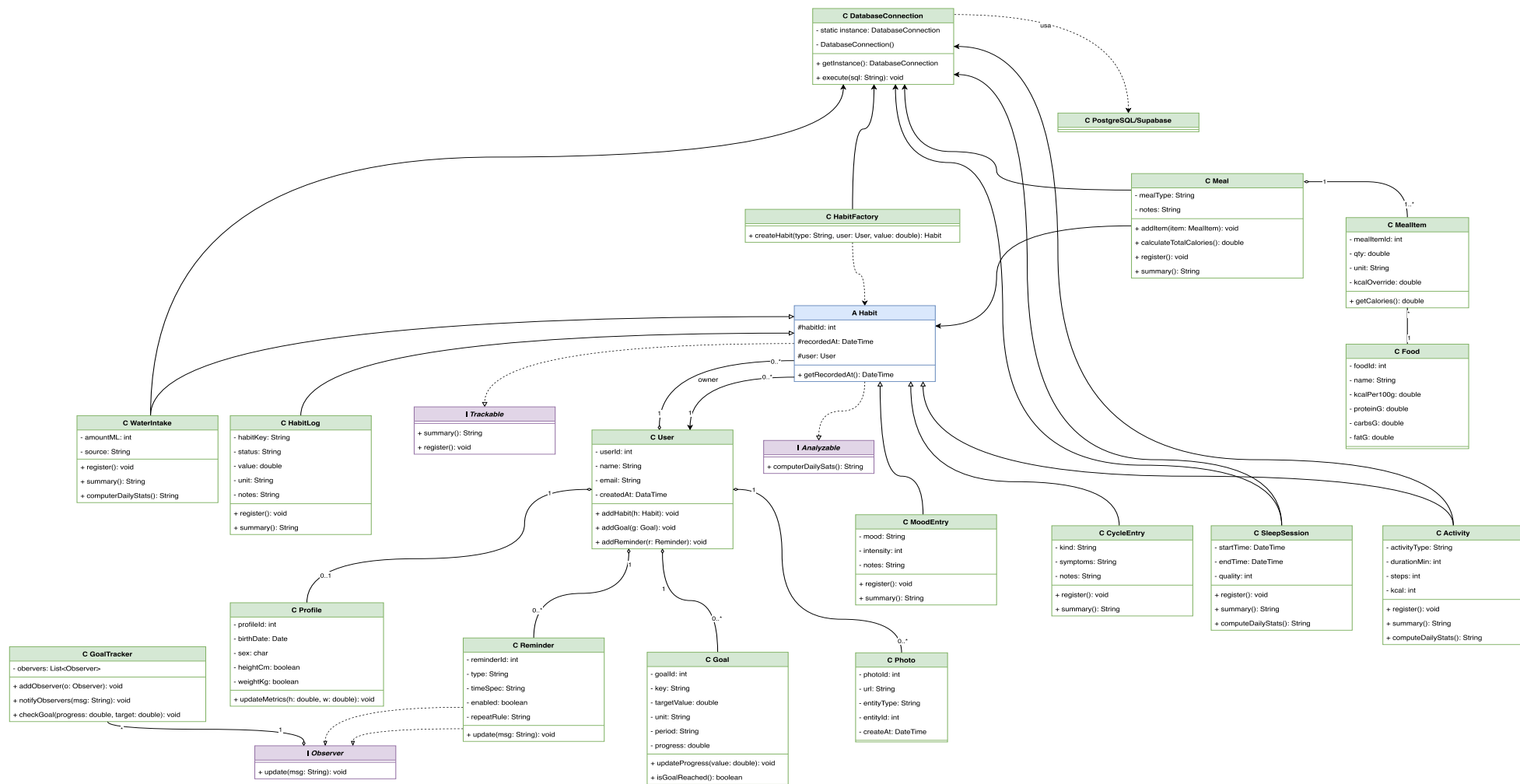


Diagrama UML



Referências bibliográficas

McKay, F. H., & Zheng, P. (2020). Behaviour change through digital self-tracking. *Digital Health*, 6, 1–12.

<https://doi.org/10.1177/2055207620914772>

Sardi, L., Idri, A., & Fernández-Alemán, J. L. (2021). A systematic review of gamification in mobile health applications. *Journal of Biomedical Informatics*, 113, 103681.

<https://www.scitepress.org/Papers/2022/110717/110717.pdf>

Swan, M. (2013). The quantified self: Fundamental disruption in big data science and biological discovery. *Big Data*, 1(2), 85–99.

<https://doi.org/10.1089/big.2012.0002>

Wolf, G., & Kelly, K. (2007). The quantified self. *Wired Magazine*.

<https://quantifiedself.com/blog/what-is-the-quantified-self/>

Materiais pedagógicos

Materiais pedagógicos das Unidades Curriculares de Programação Orientada a Objetos, Programação de Dispositivos Móveis e Bases de Dados. (2026). *Licenciatura em Engenharia Informática – EaD, Universidade Europeia*.